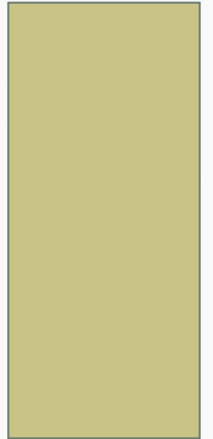


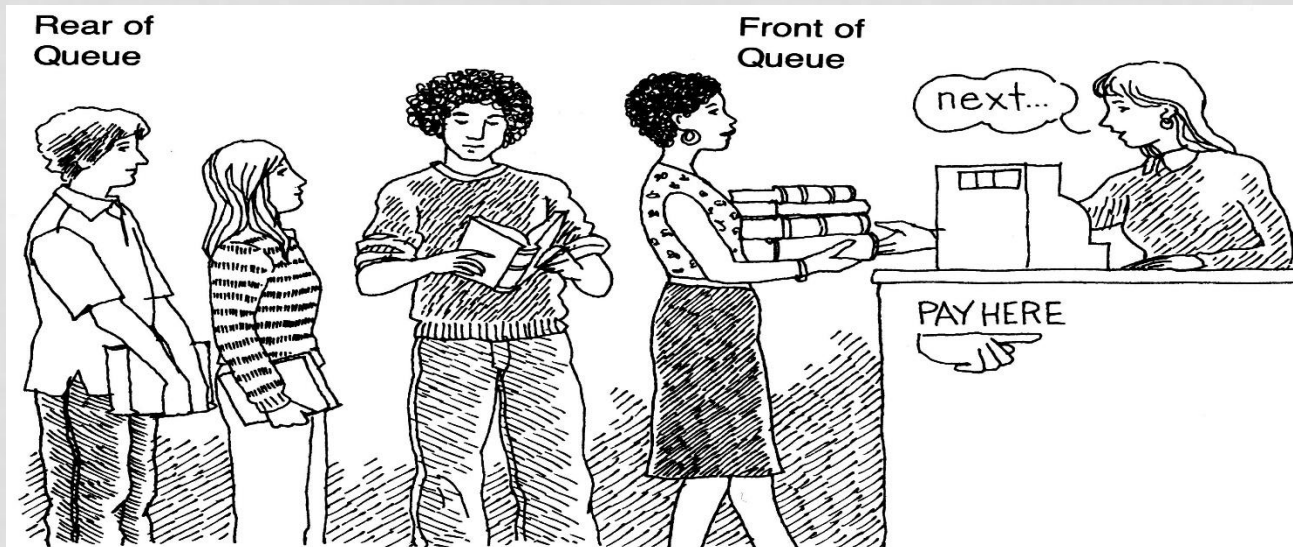
EC-200 DATA STRUCTURES

LECTURE 7
WALLS AND MIRRORS: CHAPTER 7



QUEUE

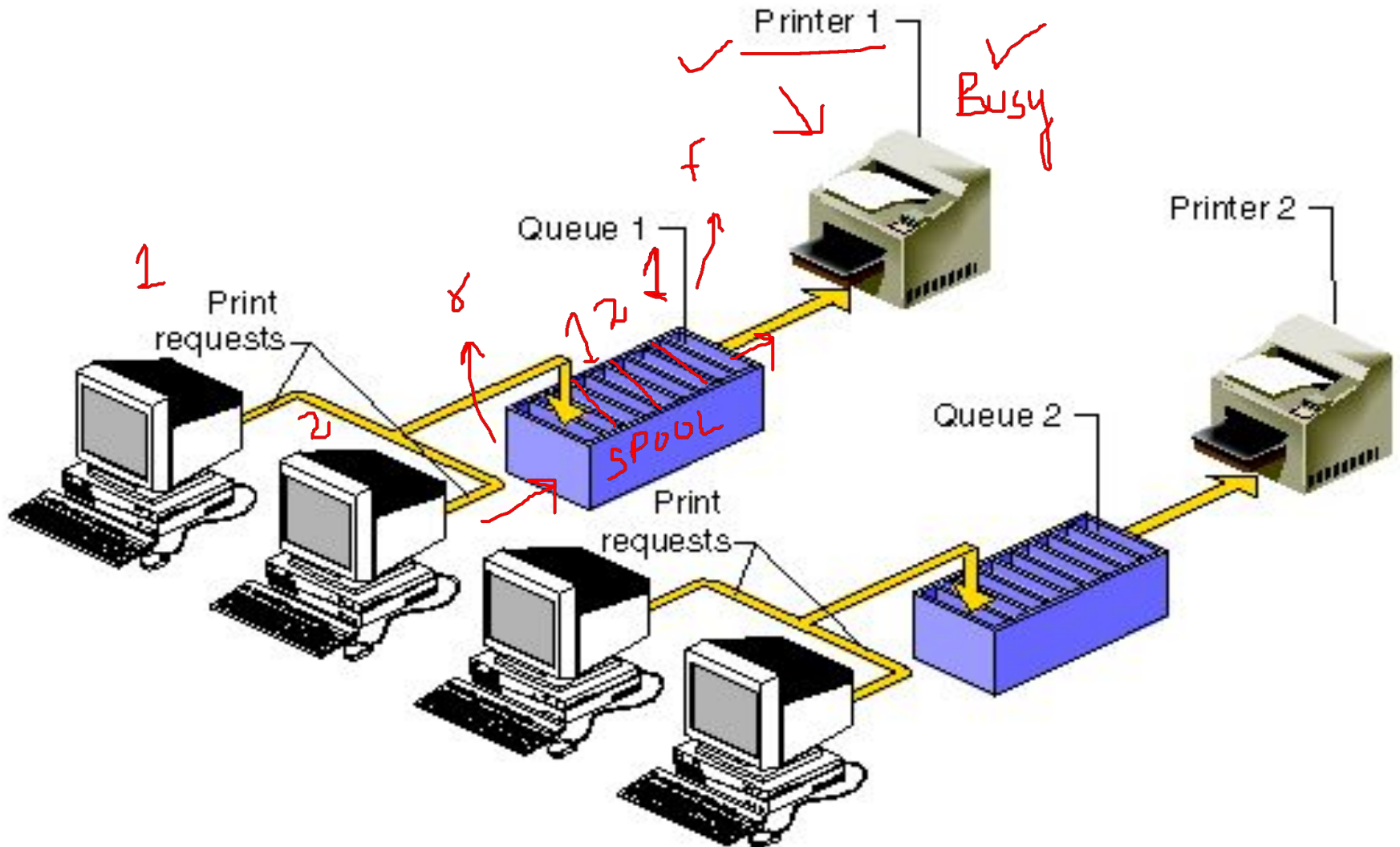
- An ordered group of homogeneous items or elements.
- Queues have **two ends**:
 - Elements are added at one end (**Rear**).
 - Elements are removed from the other end (**Front**).
- The element added first is also removed first (**FIFO**: First In, First Out).



APPLICATIONS OF THE QUEUE

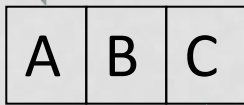
- ✓ • Printer Spooler ✓
- CPU Job Scheduling
- Event Simulation (e.g. bank counter)
- Airline reservation

PRINTER SPOOLER



WORKING EXAMPLE OF A QUEUE

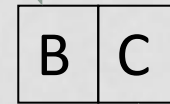
Front



Rear

(a) Queue with 3 elements

Front



Rear

(b) After removing 1 element

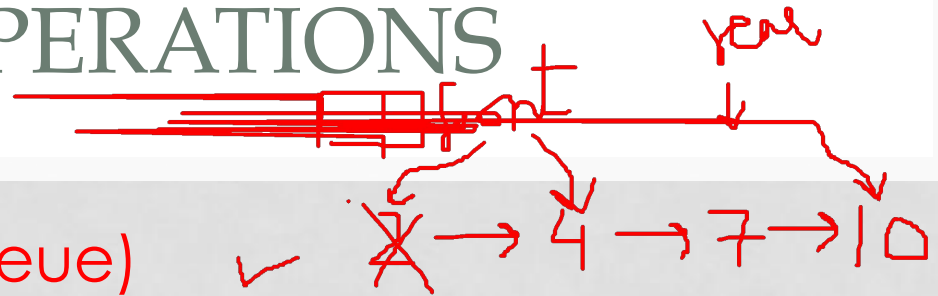
Front



Rear

(c) Inserting 2 more elements to the queue shown in part (a)

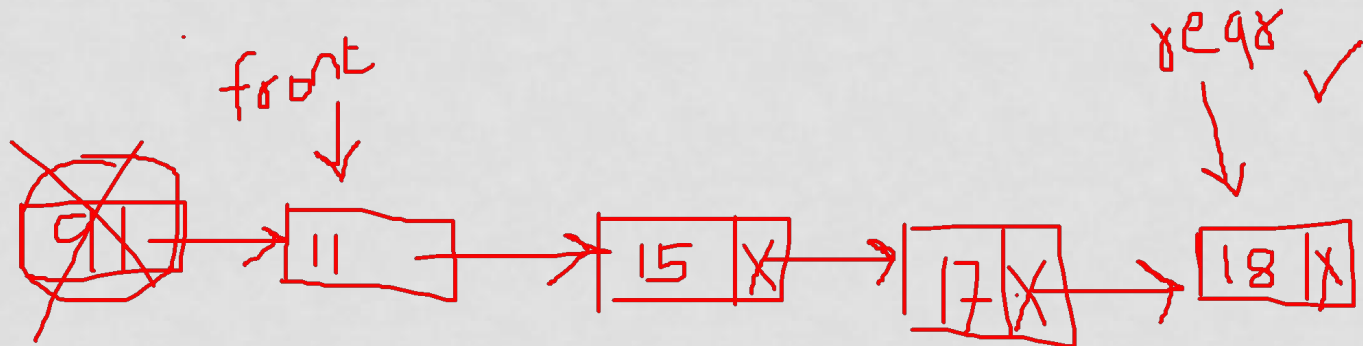
BASIC OPERATIONS



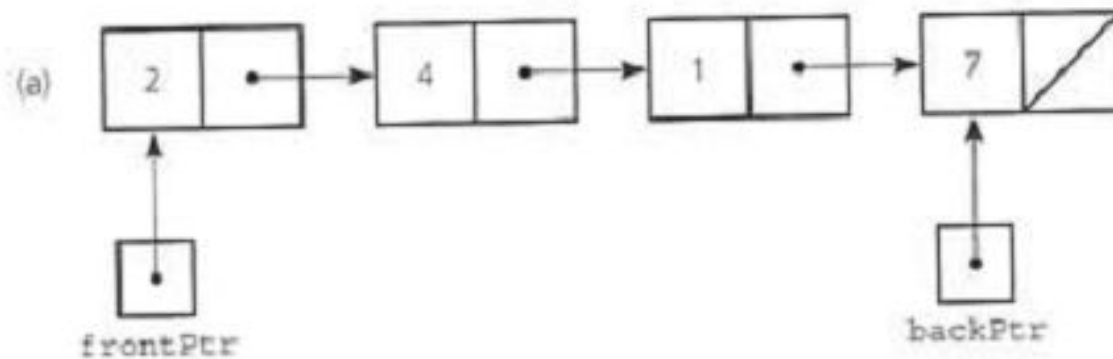
- Insert (also called Enqueue)
 - inserts item at the rear of queue.
- Remove (also called Dequeue) ✓
 - deletes the front element of the queue
 - Attempt to remove an element from an empty queue is called an underflow.
- IsEmpty ✓
 - returns false or true depending on whether or not the queue contains any elements

QUEUE IMPLEMENTATION: → DYNAMIC (POINTER BASED)

- Similar to Linked List Operations
 - **Enqueue**: same logic as InsertAtEnd() of linked list
 - **Dequeue**: same logic as RemoveFromStart() of linked list



POINTER BASED IMPLEMENTATION OF QUEUE



INSERTION OF AN ITEM

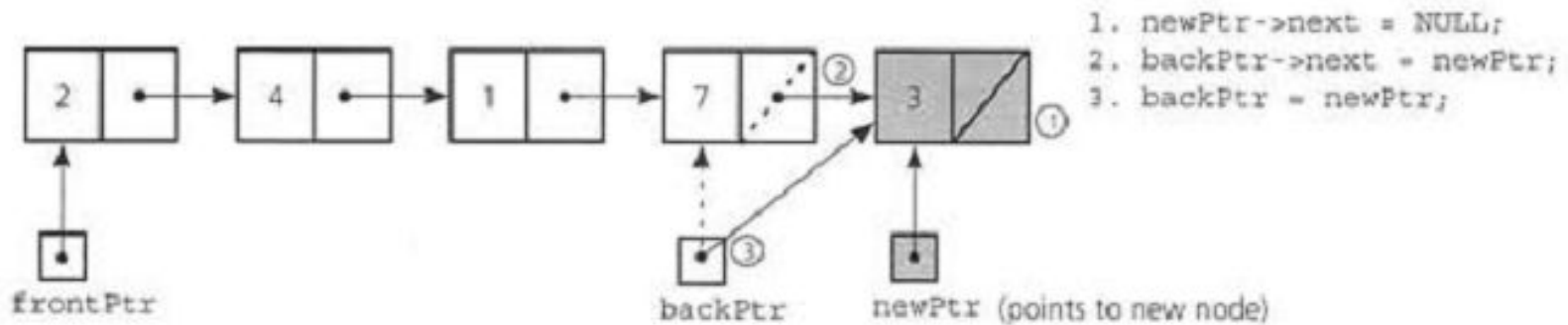


FIGURE 7-5

Inserting an item into a nonempty queue

INSERTION OF AN ITEM CONTD.

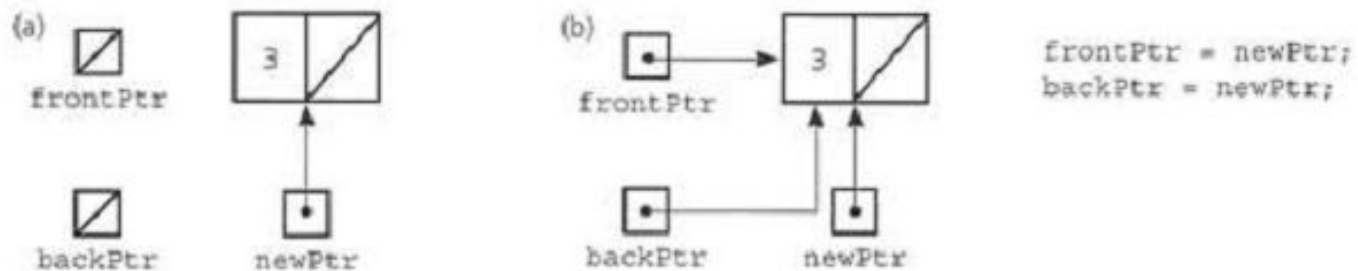


FIGURE 7-6

Inserting an item into an empty queue: (a) before insertion; (b) after insertion.

DELETION OF AN ITEM

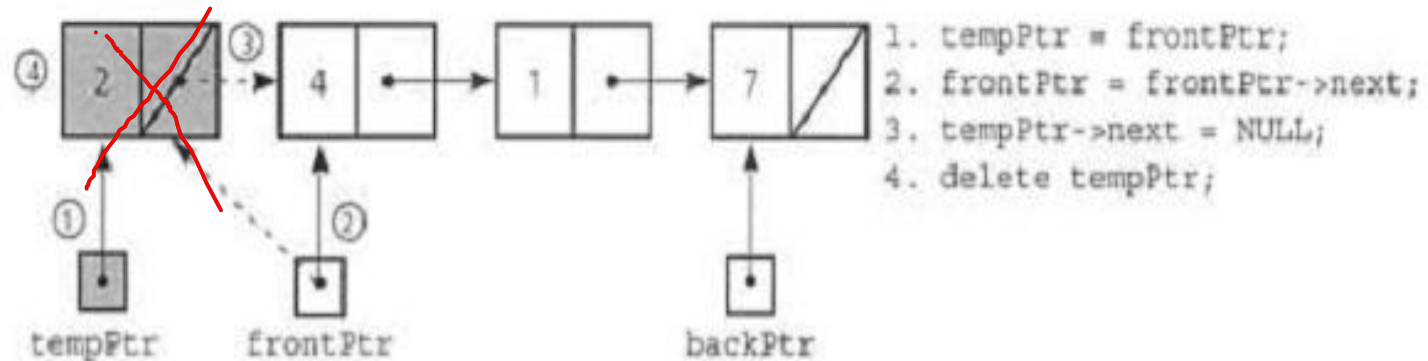


FIGURE 7-7

Deleting an item from a queue of more than one item

POINTER BASED IMPLEMENTATION OF A QUEUE

```
struct Node  
{  
    int item;  
    Node * next;  
};
```

DYNAMIC IMPLEMENTATION OF A QUEUE

```
class Queue {  
public:  
    → Queue();  
    bool isEmpty();  
    void enqueue(int newItem);  
    void dequeue();  
    void deque(int & queueFront);  
    void getFront(int & queueFront);  
    → Queue(const Queue & c);  
    → ~Queue( );  
private:  
    Node * front;  
    Node * rear; };
```

ARRAY BASED IMPLEMENTATION

```
const int MAX_QUEUE = 100;
class SQueue
{
    SQueue (); //Constructor
    bool IsEmpty(); //returns true if queue is empty
    void enqueue(int newItem); //inserts newItem at last
    void dequeue(); //removes item from front
    int getFront();
    private:
    int items [MAX_QUEUE];
    int front, rear;
};
```

STATIC QUEUE ADT

void main()

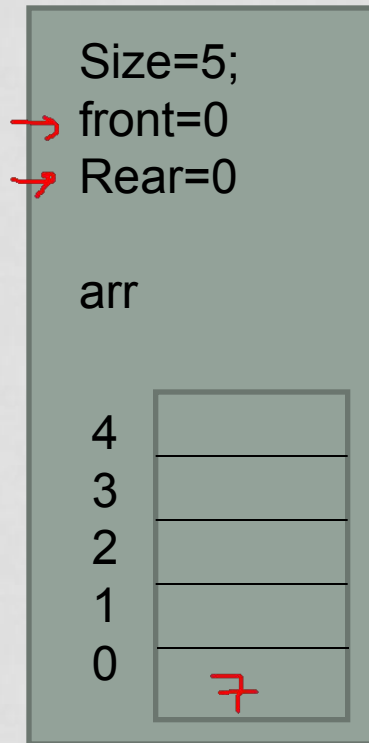
```
{SQueue<int> s;  
s.enqueue (6);  
s.enqueue(8);  
s.enqueue(15);  
s.enqueue(18);  
s.dequeue();  
s.dequeue();
```

```
s.enqueue(21);  
s.enqueue(35);  
s.enqueue(45);  
s.enqueue(48);  
s.dequeue();  
s.dequeue();  
s.dequeue();  
s.dequeue();  
s.dequeue();  
}
```

STATIC QUEUE ADT

SQueue<int> s;

s.enqueue(7)

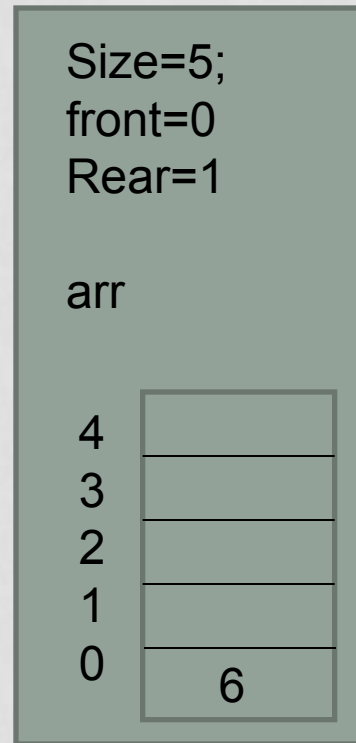


front rear = 1
↓



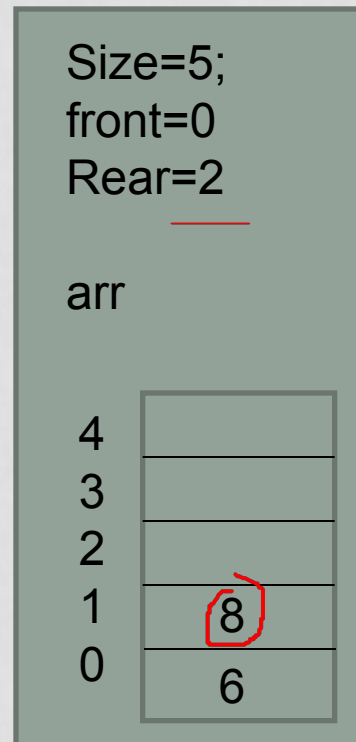
STATIC QUEUE ADT

s.enqueue(6);



STATIC QUEUE ADT

s.enqueue(8);



STATIC QUEUE ADT

s.enqueue(15);

Size=5;
front=0
Rear=3

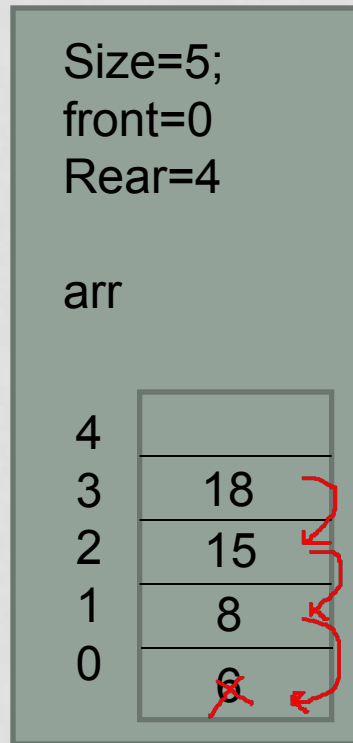
arr

4	
3	
2	15
1	8
0	6

STATIC QUEUE ADT

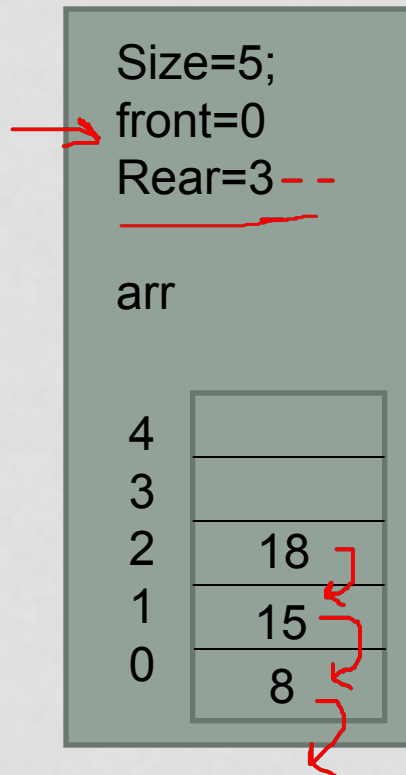
s.enqueue(18);

s.dequeue()



STATIC QUEUE ADT

s.dequeue();



STATIC QUEUE ADT

s.dequeue();

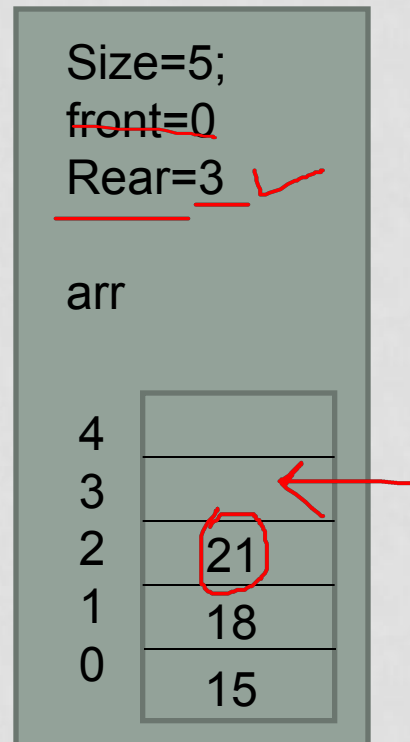
Size=5;
front=0
Rear=2

arr

4	
3	
2	
1	18
0	15

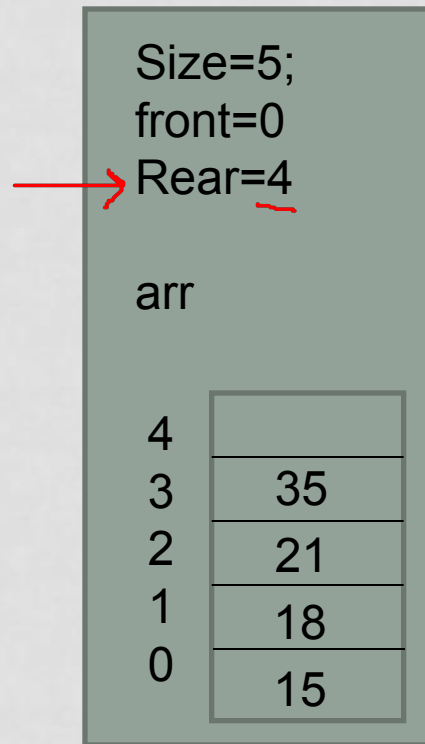
STATIC QUEUE ADT

s.enqueue(21);



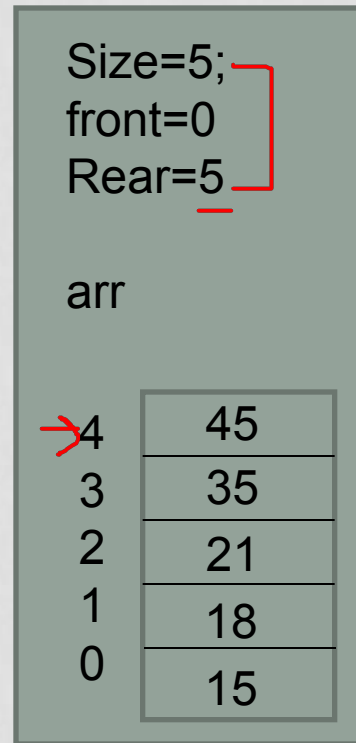
STATIC QUEUE ADT

s.enqueue(35);



STATIC QUEUE ADT

s.enqueue(45);



STATIC QUEUE ADT

s.enqueue(48);

Queue.exe

Queue overflow

Size=5;
front=0
Rear=5

arr

4	45
3	35
2	21
1	18
0	15

STATIC QUEUE ADT

s.dequeue();

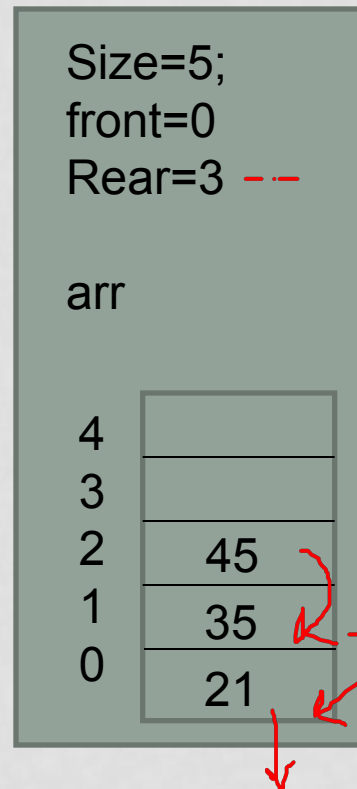
Size=5;
front=0
Rear=4

arr

4	
3	45
2	35
1	21
0	18

STATIC QUEUE ADT

s.dequeue();



STATIC QUEUE ADT

s.dequeue();

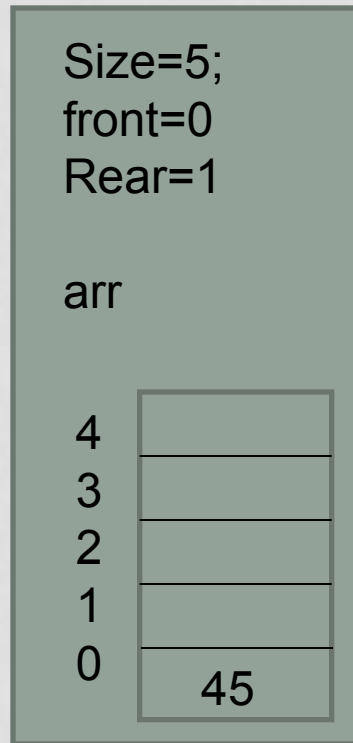
Size=5;
front=0
Rear=2

arr

4	
3	
2	
1	45
0	35

STATIC QUEUE ADT

s.dequeue();



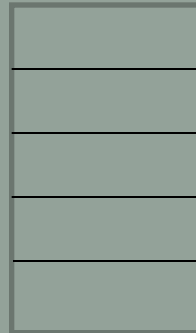
STATIC QUEUE ADT

s.dequeue();

Size=5;
front=0
Rear=0

arr

4
3
2
1
0

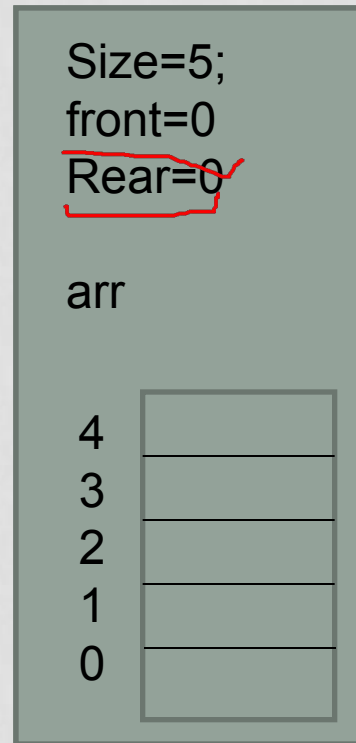


STATIC QUEUE ADT

s.dequeue();

Console

```
Queue.exe
Queue overflow
Queue underflow
```



WHAT'S THE OVERHEAD

s.dequeue();

Whenever an element is de-queued, all elements are required to be shifted one position backward to fill in the empty space. This involves an overhead.

Solution? Static Circular Queue

Size=5;
front=0
Rear=5

arr

4	45	↓
3	35	↓
2	21	↓
1	18	↓
0	15	↓

CIRCULAR STATIC QUEUE ADT

void main()

```
{CSQueue<int> s;  
s.enqueue (6);  
s.enqueue(8);  
s.enqueue(15);  
s.enqueue(18);  
s.dequeue();  
s.dequeue();
```

```
s.enqueue(21);  
s.enqueue(35);  
s.enqueue(45);  
s.enqueue(48);  
s.dequeue();  
s.dequeue();  
s.dequeue();  
s.dequeue();  
s.dequeue();  
}
```

CIRCULAR STATIC QUEUE ADT

CSQueue<int> s;

Size=5; —

front=0 —

Rear=0 —

arr

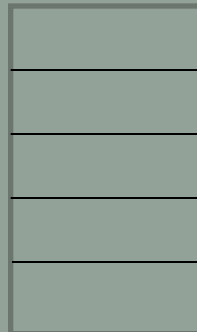
4

3

2

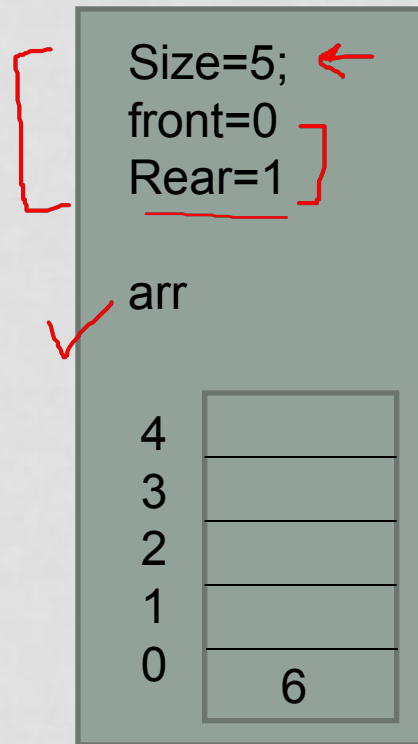
1

0



CIRCULAR STATIC QUEUE ADT

s.enqueue(6);



CIRCULAR STATIC QUEUE ADT

s.enqueue(8);

Size=5;
front=0
Rear=2

arr

4	
3	
2	
1	8
0	6

CIRCULAR STATIC QUEUE ADT

s.enqueue(15);

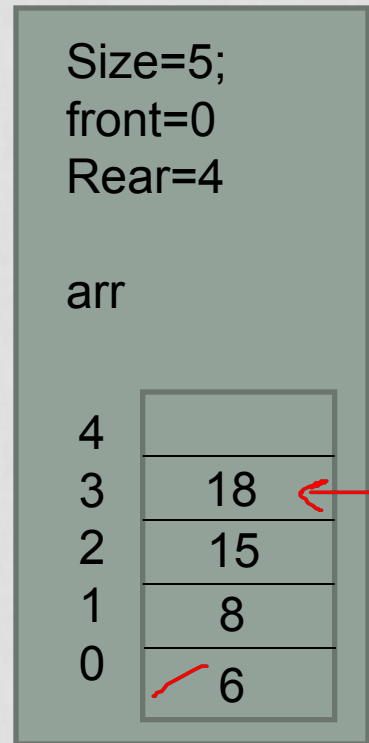
Size=5;
front=0
Rear=3

arr

4	
3	
2	15
1	8
0	6

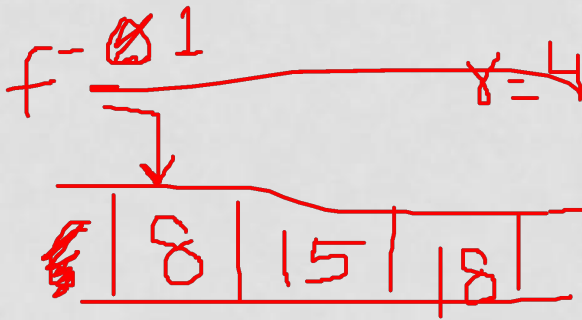
CIRCULAR STATIC QUEUE ADT

s.enqueue(18);



CIRCULAR STATIC QUEUE ADT

s.dequeue();



Size=5;
front=1
Rear=4

arr

4	
3	18
2	15
1	8
0	X

CIRCULAR STATIC QUEUE ADT

s.dequeue();

$$(rear + 1) \% 5$$

$$(6 + 1) \% 5 = 1$$

$$(0 + 1) \% 5 = 1$$

$$(4 + 1) \% 5 = 0$$

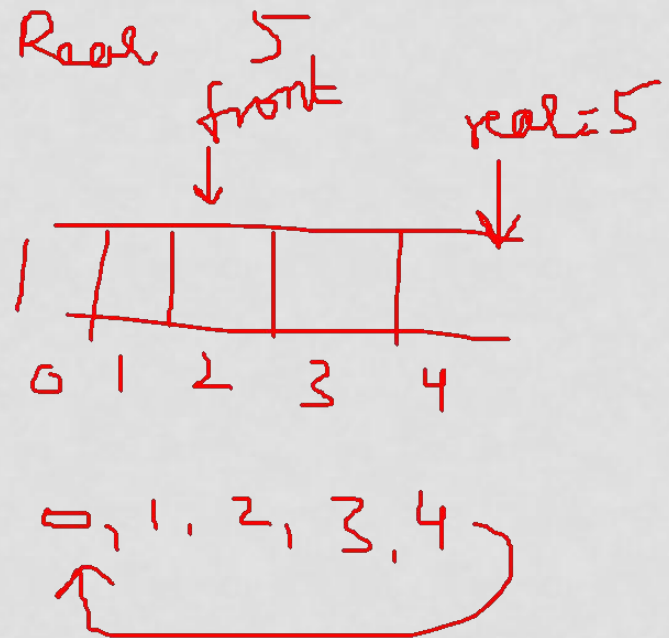
Size=5;

front=2

Rear=4

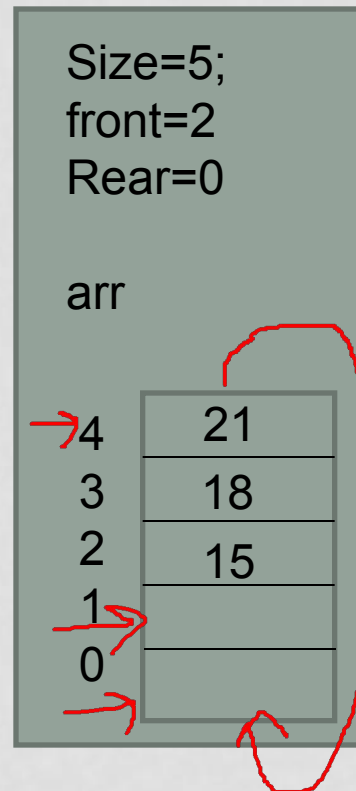
arr

4	17
3	18
→ 2	15
1	
0	



CIRCULAR STATIC QUEUE ADT

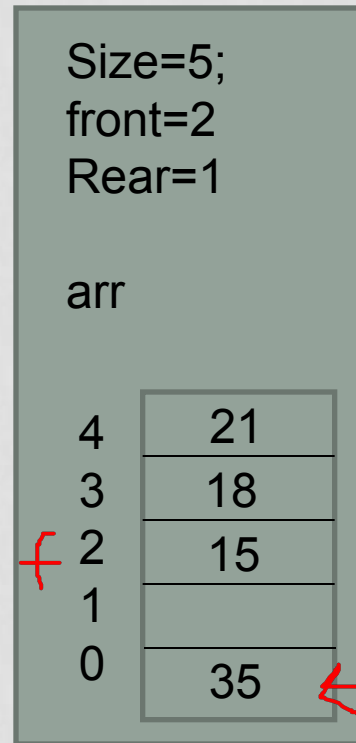
s.enqueue(21);



0, 1, 2, 3, 4,

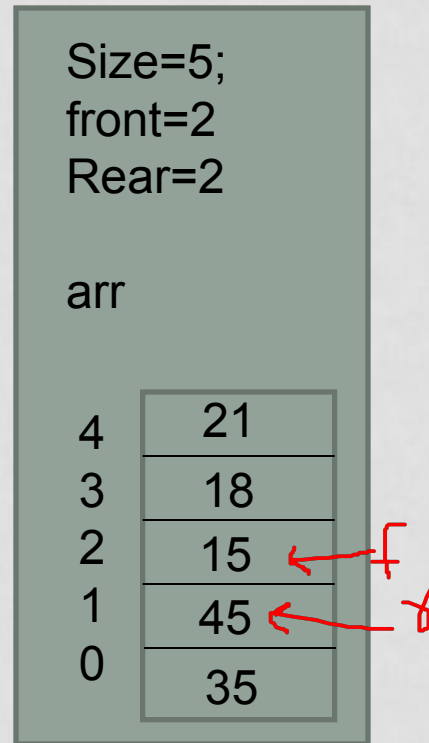
CIRCULAR STATIC QUEUE ADT

s.enqueue(35);



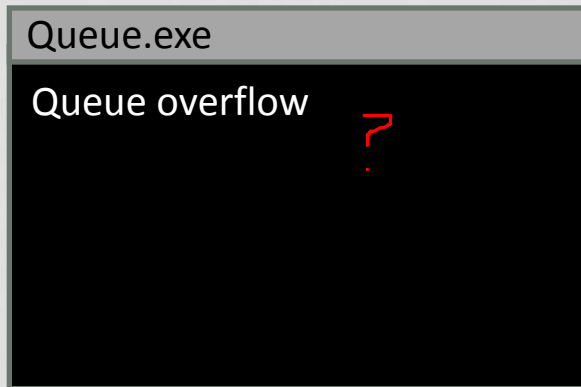
CIRCULAR STATIC QUEUE ADT

s.enqueue(45);



CIRCULAR STATIC QUEUE ADT

s.enqueue(48);

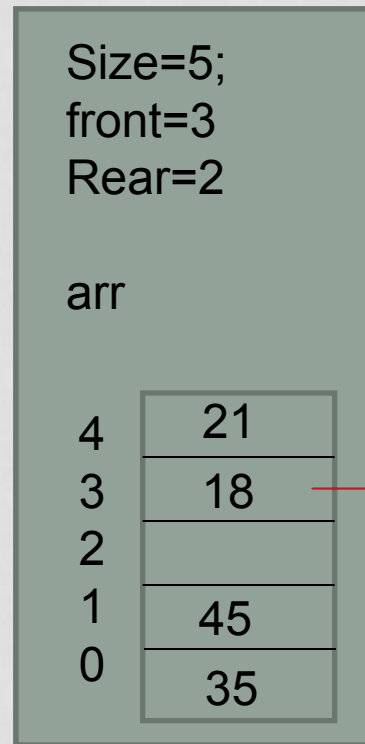


Size=5; front=2 Rear=2	
arr	
4	21
3	18
2	15
1	45
0	35

Diagram illustrating the state of a Circular Static Queue ADT. The queue is represented by an array 'arr' of size 5. The current state is: Size=5, front=2, Rear=2. The array contains the following values: index 4: 21, index 3: 18, index 2: 15, index 1: 45, index 0: 35. Red handwritten annotations show 'f' pointing to index 2 and 'r' pointing to index 1.

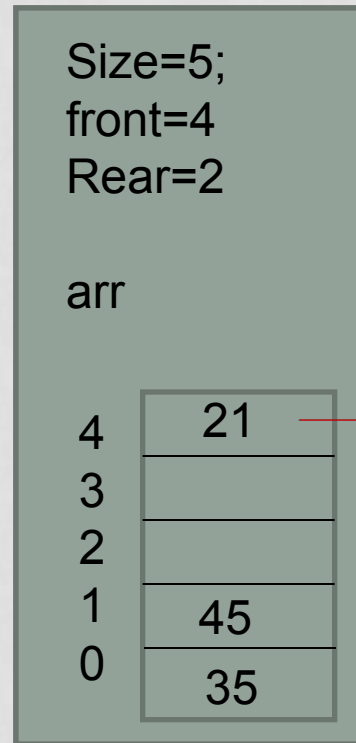
CIRCULAR STATIC QUEUE ADT

s.dequeue();



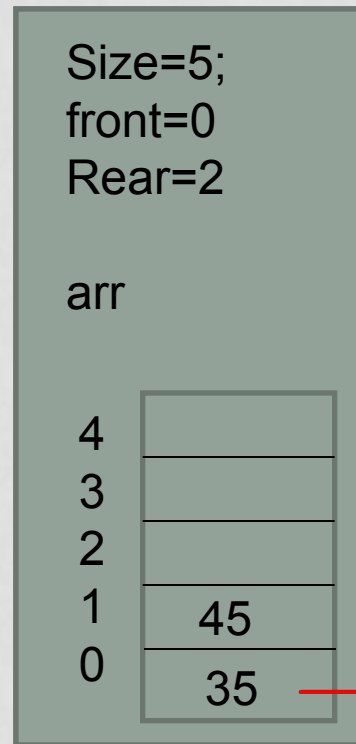
CIRCULAR STATIC QUEUE ADT

s.dequeue();



CIRCULAR STATIC QUEUE ADT

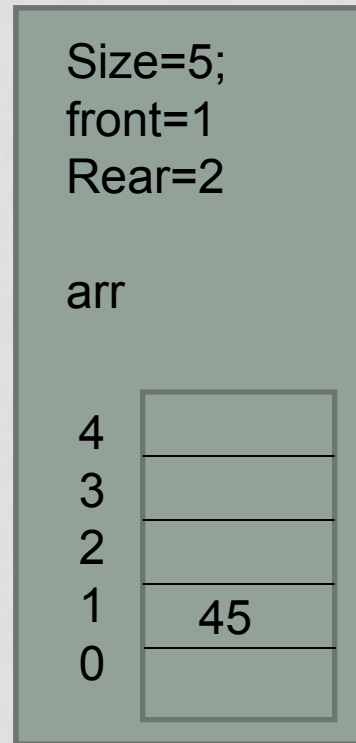
s.dequeue();



0, 1, 2, 3, 4

CIRCULAR STATIC QUEUE ADT

s.dequeue();



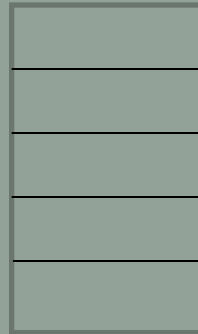
CIRCULAR STATIC QUEUE ADT

s.dequeue();

Size=5;
front=2
Rear=2

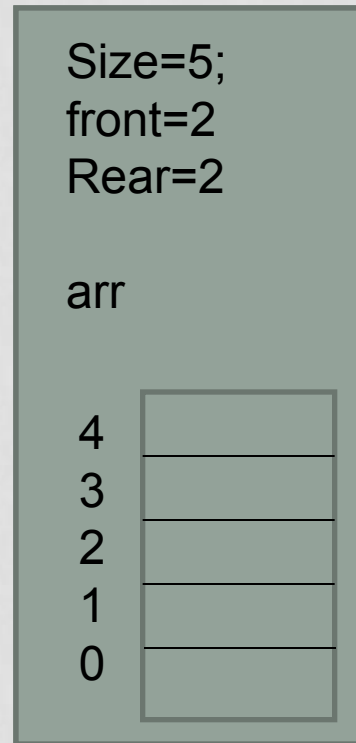
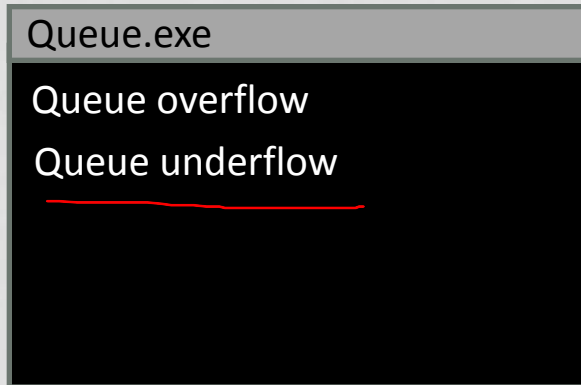
arr

4
3
2
1
0

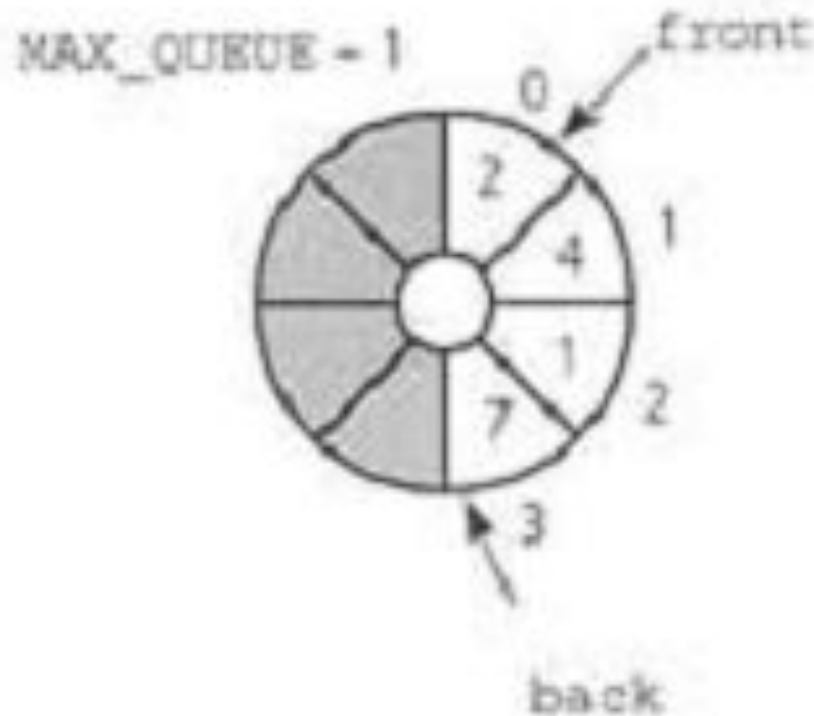


CIRCULAR STATIC QUEUE ADT

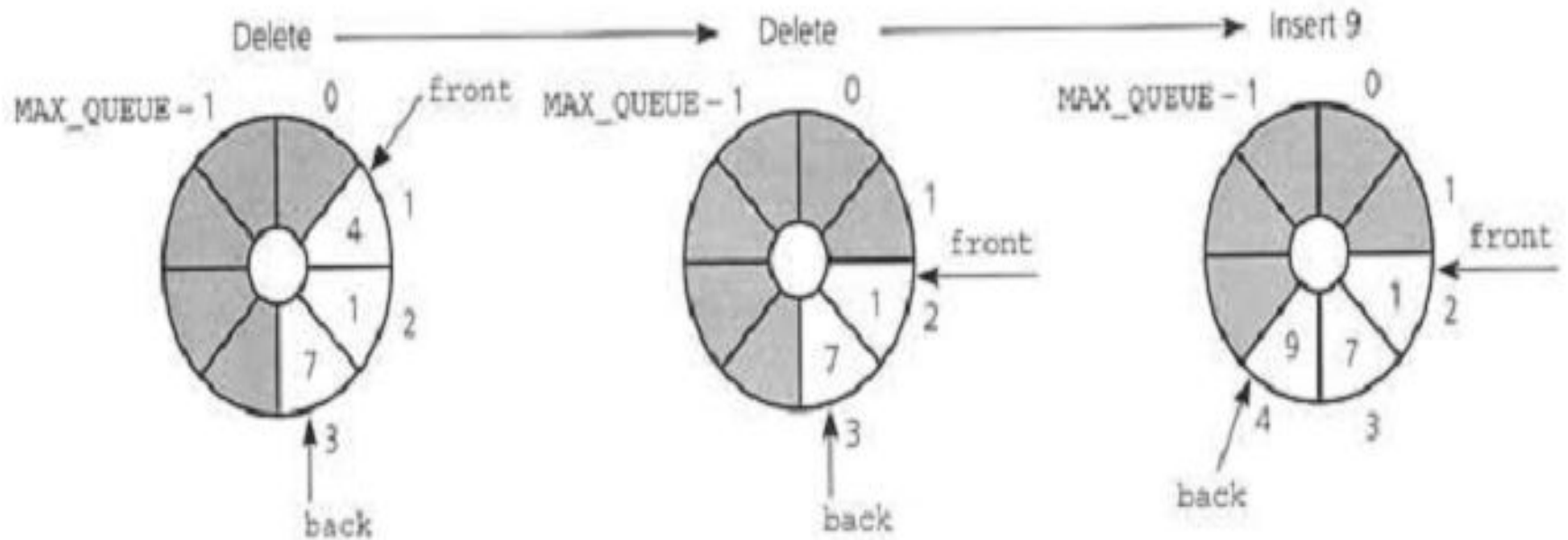
s.dequeue();



STATIC (CIRCULAR ARRAY BASED) IMPLEMENTATION OF QUEUE



EFFECT OF INSERT AND DELETE



EXAMPLE: READING A STRING OF CHARACTERS

- When You Enter Characters at a Keyboard, the System Must Retain Them in the Order in Which you Typed Them.
- It Could Use a Queue for This Purpose.

EXAMPLE: READING A STRING OF CHARACTERS

```
//read a string of characters from a single line of input into a queue
Queue Q;
while ( not end of line)
{
    Read a new character Ch;
    Q.enqueue(Ch);
}
```

Once the characters are in a queue, the system can process them as necessary

EXAMPLE: RECOGNIZING PALINDROMES

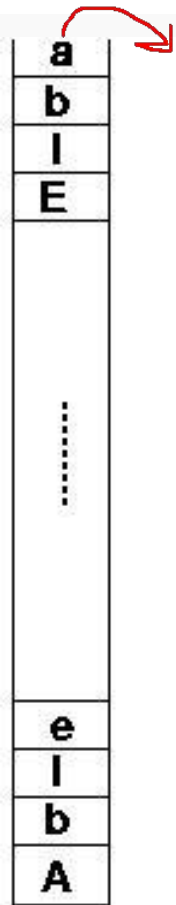
- A *palindrome* is a string that reads the same forward and backward.

able was I ere I saw elba

- We will read the line of text into both a **stack** and **a queue**.
- Compare the contents of the stack and the queue character-by-character to see if they would produce the same string of characters.

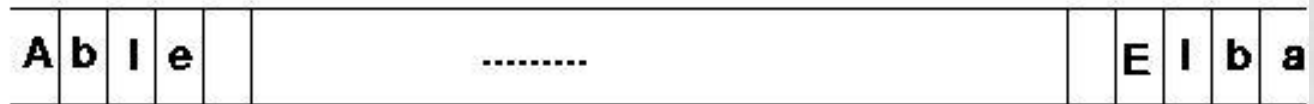
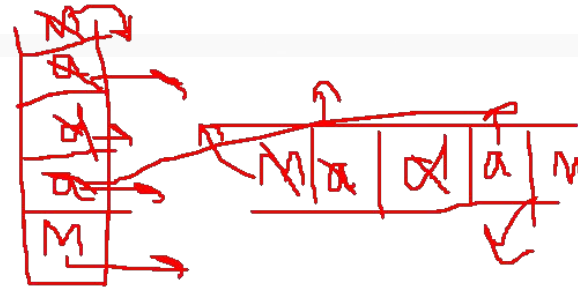
madam

EXAMPLE: RECOGNIZING PALINDROMES

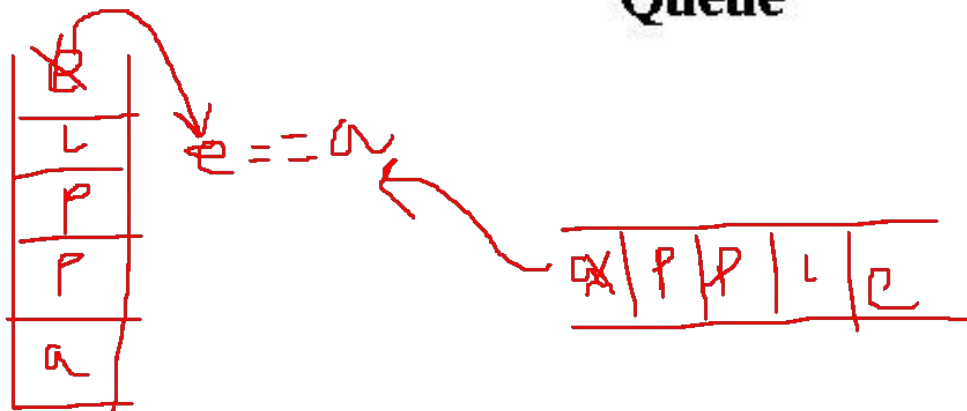


Stack

→ Madam
← Apple



Queue



READING

- Chapter 7, Data Abstraction and Problem Solving
~~using C++~~, Walls & Mirrors 3rd ed.

HELP

- <https://www.youtube.com/watch?v=FzhKQxiqVvk>